

RISC-V ISA Profile of the Upstream XMSS Reference Implementation

Wrenna Robson*

April 2026

1 Context

XMSS (RFC 8391) is a stateful hash-based signature scheme standardised for post-quantum use. An authoritative C implementation is published by the RFC authors at [XMSS/xmss-reference](#) and is widely used as a reference companion to the specification.

This report characterises that reference implementation on RISC-V 64: which ISA extensions its compiled code actually uses, where in the code base each extension appears, and how the profile changes when the Zbb bit-manipulation extension is enabled.

The analysis is purely empirical. The reference sources are cross-compiled to RV64 object files, disassembled, and every instruction is classified by looking up its mnemonic in a table generated from the `riscv-opcodes` database (the same database used by the RISC-V toolchain).

The guiding question is: *what ISA surface does a straightforward C implementation of XMSS actually exercise on RISC-V?* The answer informs decisions about minimum ISA targets, hardware acceleration opportunities, and where ISA-specific work (e.g. hand-written rotation instructions) would pay off.

2 Methodology

2.1 Analysis target

We analyse the static archive `libxmss-ref.a`, built from the reference's `SOURCES_FAST` source set, which is the efficient stateful-signing variant using the BDS tree-traversal algorithm:

*With Claude (Anthropic), which co-authored the analysis tooling and report text.

Source file	Role
<code>params.c</code>	Parameter-set derivation (wire-format OIDs)
<code>hash.c</code>	Hash wrappers (PRF, F, H, H_msg, core_hash)
<code>fips202.c</code>	Keccak / SHAKE128 / SHAKE256
<code>hash_address.c</code>	ADRS structure manipulation
<code>wots.c</code>	WOTS+ key generation, sign, verify
<code>xmss.c</code>	Top-level XMSS / XMSS-MT API (OID dispatch)
<code>xmss_core_fast.c</code>	XMSS / XMSS-MT core with BDS state
<code>xmss_commons.c</code>	L-tree, auth-path computation, shared utilities
<code>utils.c</code>	Integer-to-bytes conversion

`randombytes.c` is excluded: it is an OS-boundary CSPRNG (`/dev/urandom` on Linux) and is not part of the XMSS algorithm.

2.2 OpenSSL delegation for SHA-2

A crucial feature of the reference implementation is that `hash.c` calls `SHA256` and `SHA512` from OpenSSL’s `libcrypto`:

```
#include <openssl/sha.h>
...
SHA256(in, inlen, out);
```

The reference’s final binary is linked with `-lcrypto`, and the actual SHA-2 compression code lives in `libcrypto`, *not* in the XMSS sources. Consequently, `libxmss-ref.a` contains **no SHA-2 compression code at all**: `hash.o` is a thin dispatch shim that calls out to external `SHA256`/`SHA512` symbols.

SHAKE128 and SHAKE256, by contrast, are bundled: the repository ships its own `fips202.c` implementation of Keccak. So the archive does contain a full Keccak- $f[1600]$ permutation.

This asymmetry is central to interpreting the profile in §3. What we are measuring is “XMSS minus `libcrypto`”: the algorithm layer plus SHAKE, but without SHA-2 internals.

2.3 Toolchain

Compiler	<code>riscv64-linux-gnu-gcc 13.3.0</code> (Ubuntu)
Flags	<code>-march=rv64gc -mabi=lp64d -O3 -Wall -Wextra</code> <code>-Wpedantic</code>
Disassembler	<code>riscv64-linux-gnu-objdump</code> (Binutils 2.42)
Lookup source	<code>third_party/riscv-opcodes</code> submodule

The `rv64gc` march is the standard general-purpose profile: `RV64I + M + A + F + D + Zicsr + Zifencei + C`. It does *not* include `Zba`, `Zbb`, or other bit-manipulation extensions.

2.4 Classification

Each disassembled instruction is classified on two orthogonal axes:

1. **Semantic extension** (what the instruction does): `I`, `M`, `A`, `F`, `D`, `Zba`, `Zbb`, etc. Determined by looking up the mnemonic in the `riscv-opcodes` database.

2. **Encoding width** (how it is encoded): 16-bit compressed (C extension) or 32-bit standard. Determined from the raw instruction byte count in the `objdump` output.

GNU `objdump` renders compressed instructions using their uncompressed aliases (`sd` not `c.sd`). A classifier that looks for `c.` prefixes would report $C = 0\%$. We detect C encoding from byte width instead.

3 Findings

3.1 Extension summary

Table 1 shows the headline result: with `-march=rv64gc`, the reference implementation uses only the **I** and **M** extensions.

Table 1: Semantic extension summary across all 9 object files.

Extension	Instructions	Unique mnemonics	Notes
I	7541	49	Base integer: loads, stores, branches, arithmetic, shifts
M	79	5	<code>mulw</code> , <code>mul</code> , <code>divuw</code> , <code>divu</code> , <code>remu</code>
A, F, D	0	0	Not present despite being enabled by <code>rv64gc</code>
Zba, Zbb, Zbs	0	0	Not part of <code>rv64gc</code> ; see §4
Zicsr, Zifencei	0	0	Reserved for kernel/fence code, absent from XMSS

98.96% of instructions are base integer (I). The remaining 1.04% are M-extension multiply/divide, which—as the per-module breakdown below will show—come entirely from compiler-generated arithmetic for index and parameter calculations.

3.2 Per-module breakdown

Table 2 gives instruction counts per object file, grouped by role.

Several features are worth noting:

- `hash.o` is conspicuously small for a “hash” module (684 instructions, 0 M). This is because it merely dispatches to SHA-256/SHA-512 in `libcrypto` (see §2.2) and to SHAKE in `fips202.o`; the actual SHA-2 compression code is not in the archive.
- `fips202.o` is large (1259 instructions, 49% C-encoded). This is where the Keccak- $f[1600]$ permutation lives; the bulk of the code is 64-bit rotations and XORs applied to the 5×5 state lanes across 24 rounds.
- `xmss_core_fast.o` dominates the algorithm layer at 3169 instructions. It contains XMSS keygen/sign/verify, XMSS-MT, and the BDS tree-traversal state machine (treehash update, retain-stack handling, auth-path construction).
- `xmss.o` (the top-level API) is only 362 instructions: it is essentially OID dispatch and forwarding into `xmss_core_fast`. It uses no M instructions.

Table 2: Per-object-file instruction counts. “M” is the M-extension count; all remaining instructions are I.

Object file	Total	M insns	C-encoded (%)
<i>Hash layer</i>			
<code>fips202.o</code>	1259	6	49%
<code>hash.o</code>	684	0	64%
<i>Algorithm layer</i>			
<code>xmss_core_fast.o</code>	3169	59	47%
<code>params.o</code>	1077	5	56%
<code>xmss_commons.o</code>	513	4	54%
<code>wots.o</code>	498	5	62%
<code>xmss.o</code>	362	0	59%
<code>hash_address.o</code>	29	0	96%
<code>utils.o</code>	29	0	58%
Total	7620	79	53%

3.3 Where M instructions come from

All 79 M-extension instructions are compiler-emitted from ordinary C expressions—none are hand-written multiplies. Table 3 shows the distribution.

Table 3: M-extension instructions by mnemonic and host object file.

Count	Mnemonic	Object files	Typical origin
68	<code>mulw</code>	<code>xmss_core_fast</code> , <code>params</code> , <code>wots</code> , <code>xmss_commons</code>	32-bit index $\times$ parameter
6	<code>mul</code>	<code>xmss_core_fast</code> , <code>fips202</code>	64-bit offset / pointer arithmetic
2	<code>divu</code>	<code>fips202</code>	SHAKE absorb/squeeze offset
2	<code>remu</code>	<code>fips202</code>	SHAKE block-length modulus
1	<code>divuw</code>	<code>params</code>	parameter derivation (h/d)

The `fips202.o` M uses (`divu/remu/mul`) come from SHAKE’s variable-length absorb/squeeze loops, which do `inlen % rate` and related byte-count arithmetic. The algorithm-layer M uses are all of the form `index * params->n` for byte offsets into signature and auth-path buffers.

3.4 Compressed encoding

53% of instructions use the 16-bit C encoding. The ratio varies from 47% (`xmss_core_fast.o`, which has more shift/mask/compare-immediate instructions that do not have compressed forms) to 96% (`hash_address.o`, which is entirely small load/store/shift sequences for ADRS word manipulation).

C encoding is handled automatically by the assembler. It affects code size but not the set of required semantic extensions.

4 Enabling Zbb

The Zbb bit-manipulation extension provides single-instruction rotations (`ror`, `rori`, `rol`), byte-reversal (`rev8`), unsigned min/max (`minu`, `maxu`), and bitwise-complement AND (`andn`). These

operations are relevant to hash functions (rotations) and to several common C idioms that the compiler can recognise (`a & ~b`; ternary min/max). Zbb is *not* part of `rv64gc`; it must be requested explicitly via `-march=rv64gc_zbb`.

4.1 Empirical comparison

Re-building the archive with `-march=rv64gc_zbb` yields the comparison in Table 4.

Table 4: Extension counts: `rv64gc` vs `rv64gc_zbb`.

Extension	<code>rv64gc</code>	<code>rv64gc_zbb</code>	Delta
I	7541	6982	−559
M	79	79	0
Zbb	0	132	+132
Total	7620	7193	−427

Enabling Zbb removes 559 base-integer instructions and introduces 132 Zbb instructions, for a net decrease of 427 instructions (5.6% of the archive). The compiler has recognised that multi-instruction rotation, `not/and`, and branch-based min/max sequences can be collapsed into single Zbb ops.

Table 5 breaks down the 132 Zbb instructions by mnemonic.

Table 5: Zbb instructions emitted with `-march=rv64gc_zbb`.

Count	Mnemonic	Object files	Replaces
51	<code>andn</code>	<code>fips202</code> , <code>xmss_core_fast</code>	<code>not+and</code>
38	<code>rol</code>	<code>fips202</code>	<code>srl+sll+or</code>
20	<code>rori</code>	<code>fips202</code>	<code>srli+slli+or</code>
20	<code>maxu</code>	<code>hash</code> , <code>xmss_commons</code> , <code>xmss_core_fast</code>	branch-based max
2	<code>minu</code>	<code>xmss_core_fast</code>	branch-based min
1	<code>zext.h</code>	<code>xmss_core_fast</code>	<code>slli+srli</code> mask

4.2 Where Zbb lands

Of the 132 Zbb instructions, **108 (82%)** are in `fips202.o` (SHAKE). The 64-bit rotations (`rol`, `rori`) come directly from the Keccak- $f[1600]$ round function—the rho step rotates each lane by a fixed offset, and the iota step combines rotations with lane XORs. The `andn` instructions come from the chi step’s nonlinear function $a \oplus ((-b) \wedge c)$.

The remaining **24 (18%)** are scattered across the algorithm layer and are opportunistic: `maxu/minu` for height and index comparisons in the BDS state machine, `andn` for bitmask combinations in XMSS-MT, and a single `zext.h` for a 16-bit unsigned widening. These do not reflect any structural dependence on Zbb; the algorithm runs equally well without it.

4.3 The missing SHA-2 rotations

A conspicuous absence is `rev8` and 32-bit rotations (`rolw`, `rorw`, `roriw`). These are precisely the instructions one would expect SHA-256 / SHA-512 to use heavily—for their round-constant rotations and for big-endian conversion of the message schedule.

They are absent here for one reason: the archive does not contain SHA-2. The reference delegates SHA-256 and SHA-512 to OpenSSL’s `libcrypto` (see §2.2). Any Zbb win OpenSSL might obtain from its own SHA-2 assembly is invisible to this analysis, because `libcrypto` is not part of the measured archive.

A complete, self-contained XMSS implementation—one that ships its own SHA-256/SHA-512 in portable C—would show substantially more Zbb benefit, with a large block of 32-bit rotations and byte-reversals appearing in the hash layer. The reference’s choice to delegate SHA-2 pushes that optimisation surface into `libcrypto`, where it is architecturally opaque to clients.

5 Interpretation

5.1 Required ISA surface

With `rv64gc`, the reference implementation’s compiled footprint uses only RV64I + M. All A, F, D, Zba, Zbb, Zbs, Zicsr, and Zifencei instructions are absent. The minimum ISA to execute the code in `libxmss-ref.a` is therefore **RV64IM**. The C extension is a pure code-size win—53% of the archive is 16-bit encoded—but semantics are unchanged.

The M extension is used in only 79 of 7620 instructions (1.04%), and all 79 uses come from ordinary C arithmetic on array indices and parameter values. Neither XMSS nor WOTS+ performs large-integer multiplication: the scheme is entirely hash-based. An RV64I-only target would force the compiler to synthesise multiplies from shift and add sequences, which is feasible but unnecessary on any realistic platform.

5.2 Where hardware acceleration helps

The natural question is: where would hardware-level ISA support buy the most performance?

For this implementation, the profile says: *in the SHAKE permutation*. With Zbb enabled, 82% of the emitted Zbb instructions are in `fips202.o`. Keccak is rotation-heavy and benefits directly from single-cycle `rol/rori` and from `andn` in the chi step.

The algorithm layer benefits negligibly: 24 opportunistic Zbb uses across roughly 6000 I-extension instructions. This confirms that XMSS/WOTS+/BDS are fundamentally load/store/branch/shift-shaped workloads; there is no structural case for bitmanip acceleration at the algorithm level.

SHA-2 is invisible to this analysis by construction. For an XMSS build that ships its own SHA-2 (rather than delegating to OpenSSL), Zbb’s `rolw/roriw/rev8` would be the dominant win; the reference’s design choice simply moves that surface into `libcrypto`, where it is linked against at final-binary time.

5.3 Compiler extension inclusion is conservative

It is worth emphasising that enabling `-march=rv64gc_zbb` introduces Zbb *only where the compiler can prove equivalence*. The reference’s source code contains no RISC-V intrinsics and no architecture-specific assembly: the 132 Zbb instructions are the compiler’s unaided recognition of idiomatic C patterns $((x \gg n) \mid (x \ll (w-n)))$, `a & ~b`, `a < b ? b : a`. Deeper gains would require either hand-written intrinsics or a higher-level hash library that already exploits Zbb.

6 Reproducing this report

From the repository root, with `binutils-riscv64-linux-gnu`, `gcc-riscv64-linux-gnu`, `libssl-dev`, and the `third_party/riscv-opcodes` submodule present:

```
isa/scripts/build_reference_lib.sh rv64gc
isa/scripts/build_reference_lib.sh rv64gc_zbb
isa/scripts/analyse.sh isa/binaries/libxmss-ref-rv64gc.a \
                        isa/reports/xmss_rv64_isa_profile_reference.md
isa/scripts/analyse.sh isa/binaries/libxmss-ref-rv64gc_zbb.a \
                        /tmp/xmss_rv64_isa_profile_reference_zbb.md
```

The generated Markdown reports contain the raw per-object and per-extension tables from which Tables [1](#) through [5](#) are derived.